

AT-01

S. K. Ashwaq Ahamed

Reg No: 192572070

Sub:- Data structure

Code: CSA0306

1. An operating system manages dynamic memory dynamic memory allocation. Analyze how linked lists can represent free memory blocks. Compare this with array based memory management in terms of flexibility, fragmentation and performance, and justify the better approach.

The memory management is an important function of an operating system. It allocates memory to process and releases it after execution. A linked list is commonly used to represent free memory block. Each node in the linked list stores the starting address, size of the free block and a pointer to the next free block. When memory is allocated, the operating system finds suitable blocks and updates the linked list. When memory is released, the block is added back and adjacent free blocks are merged.

In an array-based memory management system, memory blocks are stored in fixed positions. Array blocks are stored pre-provided fast access using indexes, but insertion and deletion are difficult because elements may need to be shifted.

Array also have a fixed size, making them less flexible.

Comparison

Linked List

• Dynamic size.

• Easy insertion and deletion

• Better memory utilization

• Slightly slower access.

Array

• Fixed size.

• Difficult insertion and deletion.

• May waste memory.

• Faster access.

Advantages of linked list

• Dynamic memory allocation.

• Easy insertion and deletion of memory blocks.

• Better utilization of available memory.

• Suitable for continuously changing memory requirement.

Advantages of array

• Simple implementation

• Fast random access using index.

• Suitable when memory size is fixed

• Less pointer overhead

Justification

Linked list are suitable for operating system because they support dynamic memory allocation reduce memory wastage, and handle memory allocation and deallocation efficiently.

4 A System search records in a sorted dataset
Compare linear search and binary search using
time complexity and number of comparisons
Evaluate which is more efficient and justify
your answer

★ Searching is the process of finding a
required element in a dataset. Two common
searching methods are linear search and
Binary Search.

Linear search

It checks each element one by one until the
required element is found. It works for
both sorted and unsorted data but becomes
slow for larger dataset. Its time complexity
is $O(n)$ in the average and worst cases.

Algorithm

- starts from the first element
- compare each element with the target
- If found, return the position.
- otherwise, continue until the last element.
- If not found, report failure.

Binary Search

It works only on sorted data. It compares
the middle element and repeatedly divides
the data set into two halves until the

element is found. Its time complexity is $O(\log n)$ making it much faster than linear search.

Algorithm

- Find the middle element.
- Compare the middle element with the target.
- If equal, the search is successful.
- If the target is small, search the left half.
- If the target is larger, search from right half.
- Repeat until the element is found.

Comparison

<u>Feature</u>	<u>Linear Search</u>	<u>Binary Search</u>
• Data Required	• sorted or unsorted	• sorted only.
• Worst case	• $O(n)$	$O(\log n)$
• Comparisons	• Up to n .	About $\log_2 n$
• Speed	• Slower.	Faster.

Justification

Binary search is more efficient because it requires fewer comparisons and less execution time. Linear search may check all records, while binary search needs only about 10 comparisons. Therefore binary search is best for searching sorted dataset.